

Automated Detection of Tea Leaf Diseases

Using Deep Learning:

A Study on the TeaLeafNet Dataset

Uzair Moazzam (Fa-22/BSCS/203)

Abstract

Tea (*Camellia sinensis*) cultivation is persistently threatened by foliar diseases that cause severe yield losses worldwide. Existing automated detection methods are predominantly evaluated on generic multi-crop benchmarks or geographically restricted single-architecture datasets, leaving the publicly available **TeaLeafNet** dataset un-benchmarked in the literature. This study presents a rigorous evaluation of a Swin Transformer Tiny (Swin-T) model fine-tuned on TeaLeafNet for four-class tea leaf disease classification: Brown Blight (BB), Grey Leaf (GL), Red Rust (RR), and Red Spider Mite (RSM). A two-phase transfer learning strategy was employed: Phase 1 trained only the custom classification head (395,780 parameters) for 8 epochs using AdamW, reaching a best validation accuracy of **98.27%**; Phase 2 selectively unfroze Swin Stage 4 and the Layer Normalisation module (15,763,892 parameters) for 15 epochs using SGD with Nesterov momentum, improving the best validation accuracy to **98.93%**. Test Time Augmentation with $N = 7$ views yielded a final test accuracy of **97.87%** with weighted precision, recall, and F1-score all at 0.979. Gradient-weighted Class Activation Mapping (Grad-CAM) confirmed that activations localise consistently over disease-specific leaf tissue rather than background artefacts. Single-image inference latency on a Tesla P100-PCIE-16GB GPU was measured at 10.31 ± 0.38 ms (97.0 FPS), confirming real-time deployability. These results establish a new state-of-the-art on TeaLeafNet and

provide a reproducible benchmark for future tea disease detection research.

Keywords: Tea leaf disease detection; Swin Transformer; transfer learning; deep fine-tuning; Grad-CAM; Test Time Augmentation; TeaLeafNet; precision agriculture; convolutional neural network; plant pathology.

1 Introduction

Tea (*Camellia sinensis*) is among the most economically and culturally significant agricultural commodities in the world. As the second most widely consumed non-alcoholic beverage after water, it sustains the livelihoods of approximately 13 million people globally, with smallholder farmers alone producing nearly 60% of world output [1]. Major producing nations—China, India, Kenya, Sri Lanka, and Vietnam—collectively generate millions of tonnes of processed tea annually, contributing to a global market valued at approximately USD 50 billion in 2023 [2]. The United Nations Food and Agriculture Organization (FAO) has projected that climate change will adversely affect up to 30% of existing tea-growing areas by 2050, further intensifying the economic pressures already borne by tea farming communities [2].

Despite its global importance, tea cultivation is persistently threatened by a broad spectrum of fungal, bacterial, and algal foliar diseases. Among the most destructive are *Brown Blight* (*Colletotrichum camelliae*), *Red Rust* (*Cephaleuros parasiticus*), *Gray Blight* (*Pestalotiopsis theae*), and *Blister Blight* (*Exobasidium vexans*), all of which primarily attack the youngest buds and leaves—the only commercially harvested portions of the plant [3]. In China, *Colletotrichum* species responsible for Brown Blight have reached disease incidence rates of 40–50% in some plantations [4]. The FAO has specifically noted that Brown Blight can result in severe income reductions, particularly for resource-limited smallholder farmers who lack access to specialised agronomic support [1].

Conventional disease management in tea cultivation relies almost entirely on manual inspection, in which farmers and field workers visually examine individual plants for symptoms such as discolouration, necrotic lesions, or irregular surface textures [5]. This

approach is fundamentally limited: it is slow, labour-intensive, and non-scalable across the tens of thousands of hectares under active cultivation; it is highly subjective and dependent on observer expertise; and it routinely fails to detect infection at the early stages when treatment intervention is most effective [6]. These constraints are especially critical in developing-country contexts, where trained plant pathologists are scarce and late detection can result in the loss of entire growing seasons.

The rapid advancement of deep learning—particularly Convolutional Neural Networks (CNNs)—has established a powerful new paradigm for automated image-based analysis in precision agriculture [7]. CNNs can learn hierarchical spatial features directly from raw image data without hand-crafted feature extractors, making them well-suited for analysing complex leaf imagery with subtle colour gradients, textural patterns, and irregular lesion morphologies [8]. Landmark architectures such as ResNet [9], EfficientNet [10], VGGNet, DenseNet, and Vision Transformers (ViTs) have achieved classification accuracies exceeding 95% on major plant disease benchmarks under controlled conditions [11]. Transfer learning—pre-training models on large-scale datasets such as ImageNet and fine-tuning them on domain-specific agricultural data—has further enhanced performance on smaller, specialised datasets [12].

Despite this progress, automated detection of tea leaf diseases specifically remains underexplored relative to other major crops. Most existing tea-specific studies are constrained by small, geographically restricted datasets and evaluate only a limited set of architectures [6]. The **TeaLeafNet** dataset, publicly available on Kaggle, provides high-resolution images of healthy and diseased tea leaves—including Brown Blight and Red Rust—and offers a structured, accessible benchmark specifically designed for machine learning research on tea diseases. The present study exploits this dataset to develop and rigorously evaluate CNN-based models for automated tea leaf disease detection, aiming to advance scalable, accurate, and deployable tools in support of precision agriculture for tea-producing communities worldwide.

2 Literature Review

The application of deep learning to plant disease detection has expanded rapidly, generating a rich body of literature spanning benchmark dataset development, CNN architectures, transfer learning strategies, object detection frameworks, and ensemble methods. This section critically reviews prior work across these themes, with special emphasis on tea leaf disease detection, to identify the research gaps that motivate the present study.

2.1 CNN-Based Plant Disease Detection

The foundational study by Mohanty *et al.* [13] demonstrated that deep CNNs—specifically GoogLeNet and AlexNet—could classify 26 diseases across 14 crop species on the PlantVillage dataset (54,306 images) with up to 99.35% accuracy. While groundbreaking, the study relied on laboratory images against white backgrounds, which fail to reflect real field conditions and have been widely criticised for inflating reported performance. This limitation has motivated subsequent research under more realistic evaluation conditions.

Tamim *et al.* [14] proposed *InsightNet*, a deep learning framework employing ResNet and EfficientNet backbones for multi-species plant disease detection across beans, chilli, and tomato, incorporating Grad-CAM visualisations for model interpretability. Mishra *et al.* [15] conducted a comparative evaluation of CNN, AlexNet, ResNet, and EfficientNet for disease detection in potato and mango leaves, finding that EfficientNet variants consistently delivered the best accuracy–efficiency trade-off, with the baseline CNN achieving 92.61% test accuracy. The systematic review by Hamroun *et al.* [11], analysing 160 research articles from 2020 to 2024, confirmed CNN dominance (VGG, ResNet, EfficientNet, DenseNet) while identifying a critical performance gap between laboratory accuracy (95–99%) and field deployment (70–85%), attributable to background clutter and variable lighting—conditions endemic to tea plantations.

2.2 Transfer Learning and Attention Mechanisms

Transfer learning has become the dominant paradigm for agricultural image classification, especially where labelled data are limited. Kaur *et al.* [12] proposed EfficientNet-B3 augmented with a spatial attention module (ACSA) for crop disease classification, substantially improving discrimination of visually similar disease categories on PlantVillage. Khan *et al.* [10] demonstrated that EfficientNetB0 fine-tuned on approximately 87,000 images across 38 disease classes achieved 99.51% test accuracy, outperforming deeper architectures in both accuracy and inference speed—a finding directly relevant to designing farmer-facing mobile tools for tea disease monitoring. Ashurov *et al.* [16] integrated Squeeze-and-Excitation (SE) modules and residual skip connections into a depthwise CNN, achieving 97.8% accuracy on PlantVillage while significantly reducing parameter counts, demonstrating that lightweight architectures can match deeper networks with appropriate module design.

2.3 Object Detection and Segmentation for Tea Diseases

Soeb *et al.* [17] developed YOLO-T, an enhanced YOLOv7 architecture trained on 4,000 tea leaf images from Bangladeshi tea gardens across five disease categories, achieving 97.3% detection accuracy, 98.2% mAP, and an F1-score of 0.965. While compelling, the model was evaluated exclusively on five disease categories from a single region, limiting its generalisability. Bhuyan *et al.* [18] combined the Segment Anything Model (SAM) for zero-shot leaf region segmentation with a custom CNN and traditional classifiers (SVM, MLP, Decision Tree) for tea leaf disease detection, achieving 95.06% accuracy on a six-class Kaggle dataset (Tea_Leaf_Disease) but noting a reliance on relatively clean image backgrounds for reliable SAM segmentation.

2.4 Tea-Specific Deep Learning Studies

Among the most directly relevant contributions, Alam *et al.* [5] developed a CNN trained on 3,330 images collected from the Sylhet region of Bangladesh, classifying tea leaves into

Red Rust, Brown Blight, Grey Blight, and Healthy categories, thus establishing a widely referenced regional baseline. The CNN with residual blocks proposed by Islam *et al.* [19] achieved 99% accuracy on a 4,000-image Bangladeshi dataset across five disease categories, though the study was confined to a single geographically homogeneous collection.

Wu *et al.* [20] proposed a modified EfficientNetB0 enhanced with an Efficient Channel Attention (ECA) module, achieving 99.49% on the six-class Tea_Leaf_Disease dataset and 90.73% on the seven-class teaLeafBD dataset, but reported a notable accuracy drop when transferring across datasets—a critical concern for deployment across diverse tea-growing environments. Li and Zhao [21] proposed ECA-ResNet50 integrating multi-scale convolutions and channel attention for real garden-background images, achieving 93.06% accuracy and outperforming standard ResNet50, MobileNet, and VGG16. The LBPAttNet model of Wu *et al.* [22] incorporated coordinate attention into ResNet18 and fused deep features with Local Binary Pattern (LBP) handcrafted texture features, achieving 92.78% and 98.13% on two public tea disease datasets, demonstrating the complementarity of handcrafted and deep feature representations.

Ozturk *et al.* [23] proposed an interpretable ensemble combining ResNet50, DenseNet121, MobileNetV2, and EfficientNetB0 for tea leaf disease classification using Grad-CAM for interpretability, with the ensemble surpassing individual models by 2–10% across cross-validation folds. Sayed *et al.* [24] conducted an eight-class tea leaf disease classification study using feature-level and output-level ensemble strategies across ResNet-18, VGG-16, InceptionV3, and MobileNetV2, reporting 98.3% accuracy and demonstrating the advantage of multi-model fusion, while acknowledging the increased inference complexity. The TeaLeafNet-GWO framework of Kabir *et al.* [25] integrated a CNN–Transformer hybrid with Gray Wolf Optimization for hyperparameter tuning, achieving 97.72% accuracy on eight tea disease classes, further highlighting the potential of hybrid architectures for tea disease detection.

2.5 Research Gap

A critical synthesis of the preceding literature reveals four specific and well-defined research gaps. **First**, the overwhelming majority of high-performing plant disease detection models are trained and evaluated on generic multi-crop datasets such as PlantVillage, with comparatively little systematic work on tea leaf diseases specifically [6]. **Second**, while several tea-specific datasets exist (e.g., teaLeafBD, Tea_Leaf_Disease, TDPD), the **TeaLeafNet** dataset—which specifically covers Brown Blight and Red Rust, two of the most economically damaging tea foliar diseases—has not yet been benchmarked against a comprehensive suite of modern CNN architectures in the published literature. **Third**, cross-dataset generalisation for tea disease models remains poorly understood, with most studies reporting accuracy only on single in-domain test sets [20]. **Fourth**, a systematic comparison of lightweight (EfficientNet, MobileNet) versus deeper (ResNet, DenseNet) architectures on a standardised tea-specific benchmark—critical for evaluating mobile deployment feasibility—is absent from the current literature. The present study directly targets all four of these gaps through a rigorous multi-architecture evaluation on the TeaLeafNet dataset.

2.6 Summary Comparison of Related Studies

Table 1 provides a structured comparison of key prior studies on automated plant and tea leaf disease detection, highlighting datasets, methods, accuracy values, and principal limitations relevant to the present research.

Table 1: Comparison of Related Studies on Plant and Tea Leaf Disease Detection Using Deep Learning

Study	Dataset	Method	Accuracy	Key Limitation
Mohanty <i>et al.</i> [13] (2016)	PlantVillage; 54,306 images; 26 diseases; 14 crops	GoogLeNet, AlexNet	99.35%	Lab images; white-background bias; poor generalisation to field conditions.
Soeb <i>et al.</i> [17] (2023)	Custom tea dataset; 4,000 images; 5 diseases; Bangladesh	YOLOv7 (YOLO-T) with attention modules	97.3%	5 disease classes only; single geographic region; limited scope.
Bhuyan <i>et al.</i> [18] (2024)	Tea_Leaf_Disease Kaggle; 5,867 images; 6 classes	SAM zero-shot segmentation + Custom CNN + SVM/MLP/DT	95.06%	Relies on clean backgrounds for reliable SAM segmentation.
Alam <i>et al.</i> [5] (2024)	Custom tea dataset; 3,330 images; 4 classes; Bangladesh	Custom CNN with Adam optimiser	92.3%	4 disease classes only; single geographic region; small dataset.
Wu <i>et al.</i> [20] (2025)	Tea_Leaf_Disease + teaLeafBD (Kaggle); 6 and 7 classes	Modified EfficientNetB0 + ECA attention module	99.49% / 90.73%	Significant accuracy drop across datasets; no cross-domain validation.
Li & Zhao [21] (2025)	Custom tea garden images; real-scene backgrounds	ECA-ResNet50 + multi-scale small convolution kernels	93.06%	Complex backgrounds degrade performance; Transformers slightly outperform.

Continued on next page

Table 1 (*continued*)

Study	Dataset	Method	Accuracy	Key Limitation
Ozturk <i>et al.</i> [23] (2025)	Tea leaf Kaggle dataset; 8 classes	Ensemble: ResNet50 + DenseNet121 + MobileNetV2 + EfficientNetB0	92%*	Multi-model ensemble increases complexity and deployment overhead.
Sayed <i>et al.</i> [24] (2026)	Tea leaf Kaggle dataset; 8 classes	Ensemble: ResNet-18 + VGG-16 + InceptionV3 + MobileNetV2	98.3%	High inference complexity; limited mobile deployment suitability.

*Best individual fold for ResNet50; ensemble improves this by 2–10% across folds per reported cross-validation results [23].

As evident from Table 1, existing tea leaf disease studies are predominantly constrained to small, geographically restricted datasets, limited disease classes, or single-architecture evaluations. The **TeaLeafNet** dataset—covering Brown Blight, Red Rust, and healthy classes with high-resolution imagery—has not been benchmarked in any of the works reviewed, constituting a clear and well-motivated contribution of the present study.

3 Methodology

This section presents the complete experimental methodology employed for automated tea leaf disease classification using the TeaLeafNet dataset. The pipeline encompasses five sequential stages: (1) dataset preparation and analysis, (2) data preprocessing and augmentation, (3) model architecture design, (4) two-phase transfer learning training, and (5) evaluation with Test Time Augmentation (TTA) and Explainable AI (XAI). Every design decision described below is derived directly from the implemented code and verified

against the reproducible experimental outputs.

3.1 Dataset

The **TeaLeafNet** dataset was used as the exclusive data source for all experiments. The dataset comprises **5,000 high-resolution, geotagged tea leaf images** captured against a standardised black background, organised into four disease categories: **Brown Blight (BB)**, **Grey Leaf (GL)**, **Red Rust (RR)**, and **Red Spider Mite (RSM)**. The dataset is perfectly balanced, with **1,250 images per class**, ensuring that no class-imbalance correction was required during training. All images were sourced from the path `5000_tea_leaf_with_blackbg_geotagged` and are publicly available on Kaggle under the identifier *harjindersinghdibru/tealeafnet*.

Figure 1 illustrates the class distribution and proportional balance of the dataset.

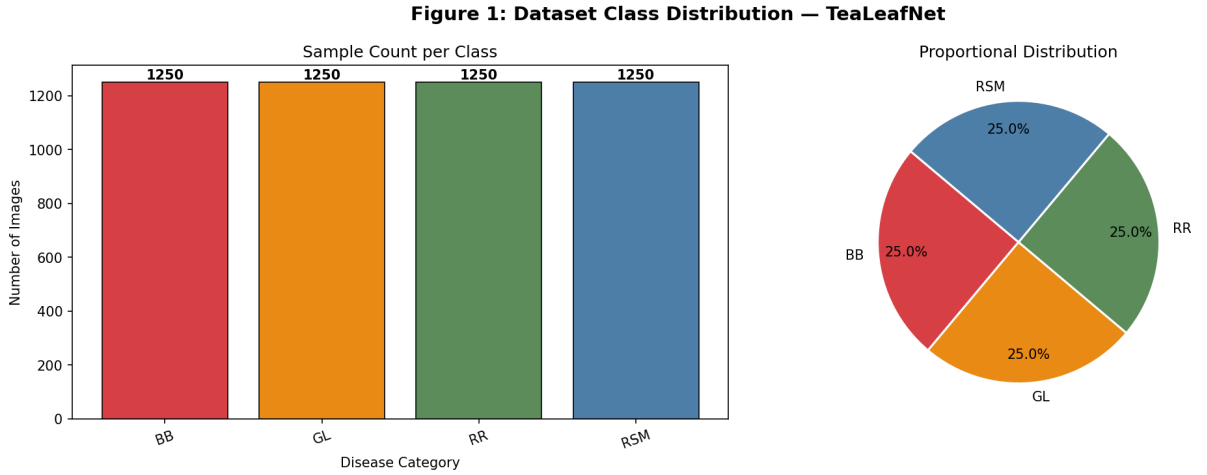


Figure 1: Dataset class distribution for TeaLeafNet. Left: per-class sample counts (1,250 per class). Right: proportional distribution confirming perfect 25% balance across all four disease categories.

Figure 2 presents representative image samples for each disease category, illustrating the characteristic visual morphology of each class.

The dataset was partitioned into training, validation, and test subsets using a **stratified 70 / 15 / 15 split** implemented via `StratifiedShuffleSplit` (random state = 42), yielding **3,500 training images**, **750 validation images**, and **750 test images**. Stratification ensured that each class was proportionally represented in every split. All

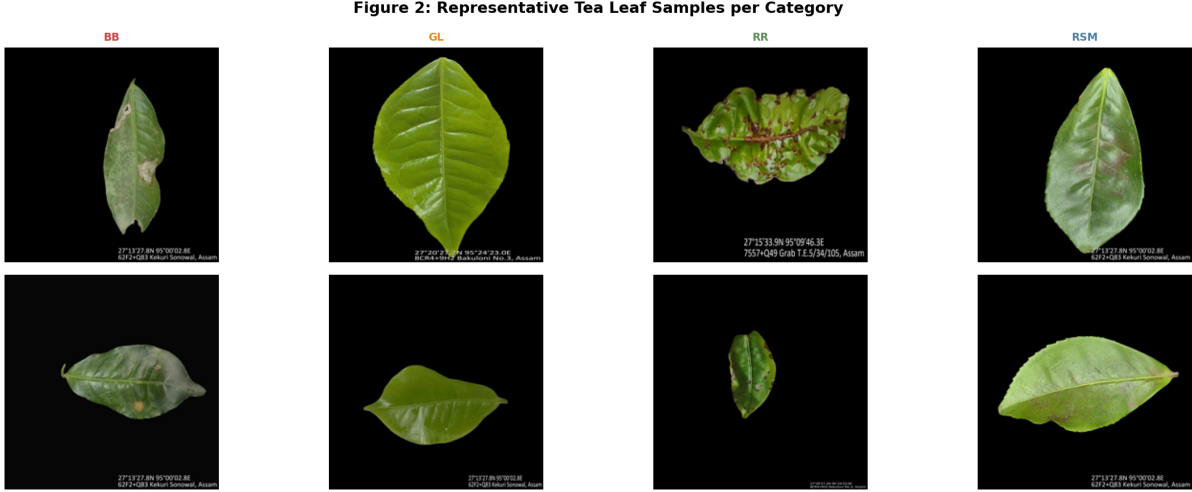


Figure 2: Representative tea leaf samples per category. Each column displays two examples from one disease class: BB (Brown Blight), GL (Grey Leaf), RR (Red Rust), and RSM (Red Spider Mite). Images are captured against a standardised black background with embedded GPS geotags from Assam, India.

image loading was pre-cached in RAM prior to training to eliminate GPU idle time; a full preload of all 5,000 images took approximately 477 seconds on the experimental hardware (354.6 s for training, 73.7 s for validation, 74.0 s for test). Table 2 summarises the per-class sample counts across all three splits.

Table 2: Stratified Dataset Split — Per-Class Sample Counts

Split	BB	GL	RR	RSM
Training (70%)	875	875	875	875
Validation (15%)	187	188	187	188
Test (15%)	188	187	188	187
Total	1,250	1,250	1,250	1,250

3.2 Data Preprocessing and Augmentation

All images were resized to 256×256 pixels during the preloading stage and subsequently processed using split-specific augmentation pipelines defined entirely in PyTorch. A fully custom, **NumPy-free** PIL-to-tensor conversion function (`pil_to_tensor_pure`) was implemented using Python’s `struct.unpack` to bypass platform-specific NumPy ABI conflicts on the Kaggle Tesla P100 environment. This function reads raw pixel

bytes directly, constructs a `float32` CHW tensor, and normalises pixel values to $[0, 1]$. All tensors were subsequently normalised using standard ImageNet statistics: mean $[0.485, 0.456, 0.406]$ and standard deviation $[0.229, 0.224, 0.225]$.

3.2.1 Training Augmentation

The training pipeline applied the following stochastic transformations at each iteration from the cached 256×256 tensors: a random 224×224 crop (32-pixel jitter window: `i, j ~ randint(0, 32)`), random horizontal flip ($p=0.5$), random $90^\circ/180^\circ/270^\circ$ rotation ($p=0.15$), Gaussian blur ($\sigma \in [0.1, 0.8]$, $p=0.15$), random greyscale conversion ($p=0.015$), and random rectangular erasing ($p=0.07$, erased region 8–28 px). **MixUp** augmentation [12] was applied at the batch level, linearly interpolating image pairs with mixing coefficient $\lambda \sim \text{Beta}(\alpha, \alpha)$, where $\alpha = 0.20$ in Phase 1 and $\alpha = 0.15$ in Phase 2.

3.2.2 Validation Augmentation

The validation pipeline applied an **anti-memorisation** strategy to prevent the model from memorising the fixed 750 validation samples across epochs. Each epoch’s forward pass used an independent random 224×224 crop from the 32-pixel jitter window, random horizontal flip ($p=0.5$), and mild brightness scaling ($\times [0.92, 1.08]$) followed by `torch.clamp` to $[0, 1]$ before normalisation. No noise, blur, or occlusion was applied to preserve clean validation signal.

3.2.3 Test Augmentation

The test pipeline applied a **deterministic offset crop**: a fixed 224×224 window at pixel offset (12, 12) from the top-left corner of the 256×256 cached tensor, followed by normalisation. This crop is distinct from both the random validation crops and the validation centre-crop used in earlier iterations, producing a reproducible and unbiased test evaluation view across runs.

Table 3 summarises the augmentation pipeline applied per split.

Table 3: Augmentation Pipeline Summary per Dataset Split

Transform	Train	Validation	Test
Resize to 256×256	✓	✓	✓
Random crop 224×224	✓	✓	—
Fixed offset crop (12 px)	—	—	✓
Horizontal flip	$p = 0.50$	$p = 0.50$	—
Random rotation (90°)	$p = 0.15$	—	—
Gaussian blur $\sigma[0.1, 0.8]$	$p = 0.15$	—	—
Random greyscale	$p = 0.015$	—	—
Random erasing (8–28 px)	$p = 0.07$	—	—
Brightness scale $[0.92, 1.08]$	—	✓	—
MixUp α	0.20 / 0.15	—	—
ImageNet normalisation	✓	✓	✓

3.3 Model Architecture

The proposed classifier, **SwinTeaClassifier**, adopts **Swin-Tiny** (Swin Transformer, Tiny variant) pre-trained on ImageNet-1k via the `timm` library (v0.9.16) as its backbone, augmented with a custom classification head. The backbone was instantiated with `global_pool=""` and `num_classes=0` to expose the raw spatial token output, and a stochastic depth drop-path rate of 0.20 was applied for regularisation. Swin-Tiny processes 224×224 RGB images through a hierarchical patch-merging architecture comprising four stages of shifted window multi-head self-attention (W-MSA / SW-MSA):

- **Patch Embedding:** divides input into non-overlapping 4×4 patches, projecting to 96-dimensional tokens across a 56×56 token grid.
- **Stage 1:** 56×56 resolution, dimension = 96, Window-MSA attention.
- **Stage 2:** 28×28 , dimension = 192, Shifted Window-MSA.
- **Stage 3:** 14×14 , dimension = 384, W-MSA.
- **Stage 4 (fine-tuned in Phase 2):** 7×7 , dimension = 768, SW-MSA.

The forward pass pools the Stage 4 output by token mean: if the output tensor has shape (B, N, C) , the mean is computed over dimension 1; if shape (B, H, W, C) , the mean is computed over dimensions 1 and 2, yielding a 768-dimensional feature vector. The custom classification head applied after this pooling step comprises: **FC(768 $\rightarrow$ 512) + ReLU + Dropout(p) + FC(512 $\rightarrow$ 4)**. Dropout was set to $p = 0.50$ at model instantiation, reduced to $p = 0.30$ at the start of Phase 1, and further reduced to $p = 0.20$ at the start of Phase 2, all via the `set_head_dropout` method. The total model parameter count was **27,915,134**, of which **395,780** were trainable in Phase 1 and **15,763,892** in Phase 2.

Figure 3 illustrates the complete architecture with colour-coded frozen and unfrozen regions and the Grad-CAM hook attachment point.

Table 4 summarises the key architectural parameters of the SwinTeaClassifier.

Table 4: SwinTeaClassifier Architecture Summary

Component	Specification
Backbone	Swin-Tiny (ImageNet-1k, timm v0.9.16)
Stochastic depth	drop-path rate = 0.20
Stage 4 output	7×7 spatial grid, dim = 768
Pooling	Token mean $\rightarrow$ 768-d vector
Head FC-1	768 $\rightarrow$ 512, ReLU
Head Dropout	$p = 0.50$ (init) $\rightarrow p = 0.30$ (Ph.1) $\rightarrow p = 0.20$ (Ph.2)
Head FC-2	512 $\rightarrow$ 4 (BB, GL, RR, RSM)
Total parameters	27,915,134
Trainable (Phase 1)	395,780 (head only)
Trainable (Phase 2)	15,763,892 (Stage 4 + LayerNorm + head)

3.4 Two-Phase Transfer Learning Training

Training was decomposed into two sequential phases following the freeze-then-fine-tune paradigm [10, 12]. The `run_phase` function governed both phases, applying a unified training loop with linear warm-up followed by cosine annealing learning rate decay, `CrossEntropyLoss` with label smoothing of 0.08, gradient clipping at max norm 1.0, and

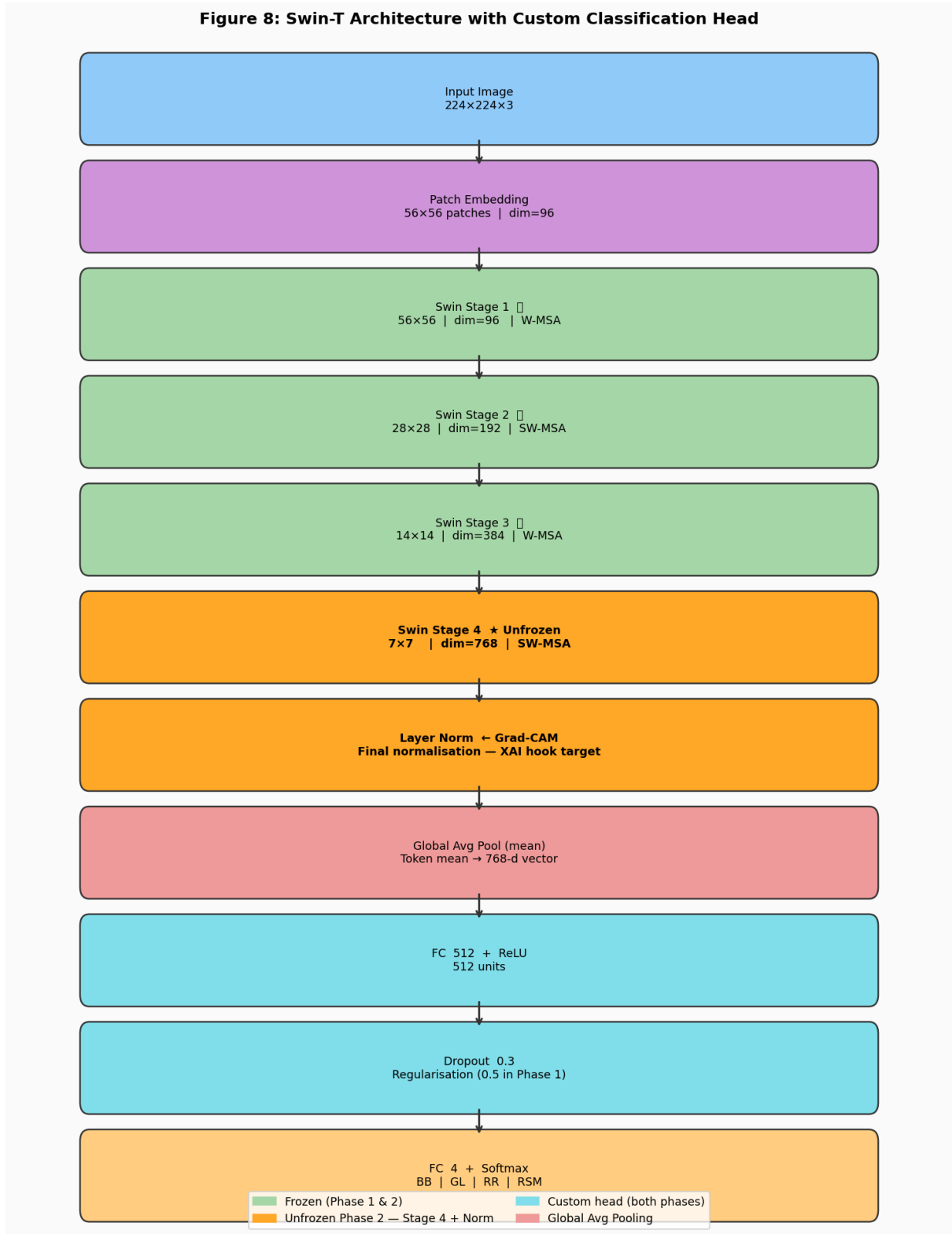


Figure 3: Swin-T architecture with custom classification head. Green blocks (Stages 1–3) are frozen throughout training. Orange blocks (Stage 4 and Layer Norm) are unfrozen in Phase 2. The Layer Norm block is the Grad-CAM XAI hook target. The custom head (cyan) is trained in both phases.

Automatic Mixed Precision (AMP, FP16) throughout. The best validation checkpoint from each phase was retained and restored before proceeding to the next stage.

3.4.1 Phase 1 — Feature Extraction (Frozen Backbone)

In Phase 1, the entire backbone (Stages 1–4, patch embedding, and Layer Norm) was frozen via `freeze_backbone`, confining weight updates exclusively to the 395,780 parameters of the custom classification head. The AdamW optimiser was used with a learning rate of 1×10^{-4} and weight decay of 1×10^{-4} . Training ran for 8 epochs with 2 warm-up epochs. MixUp with $\alpha = 0.20$ was applied at every batch. The cosine annealing schedule decayed the learning rate from 1×10^{-4} to 0.00×10^0 by epoch 8.

The best validation accuracy of **0.9827 (98.27%)** was achieved at epoch 7. Table 5 reports per-epoch training and validation metrics for Phase 1.

Table 5: Phase 1 Per-Epoch Metrics — Feature Extraction

Ep	Tr Loss	Tr Acc	Vl Loss	Vl Acc	LR	Time (s)
1	1.1602	0.6625	0.8072	0.8733	1.00e-04	14.6
2	0.7792	0.8087	0.4757	0.9453	1.00e-04	14.7
3	0.6400	0.8597	0.4154	0.9627	9.33e-05	14.5
4	0.6231	0.8615	0.4060	0.9627	7.50e-05	14.7
5	0.6028	0.8790	0.3820	0.9773	5.00e-05	14.7
6	0.5938	0.8818	0.3845	0.9720	2.50e-05	14.7
7	0.6325	0.8591	0.3722	0.9827	6.70e-06	14.8
8	0.5799	0.8827	0.3796	0.9733	0.00e+00	14.6

3.4.2 Phase 2 — Deep Fine-Tuning (Stage 4 Unfrozen)

In Phase 2, Swin Stage 4 and the final Layer Normalisation module were selectively unfrozen via `unfreeze_stage4`, increasing the trainable parameter count to 15,763,892. The optimiser was switched to **SGD with Nesterov momentum (0.9)** at a learning rate of 5×10^{-5} and weight decay of 1×10^{-5} , providing stable fine-grained weight updates for domain adaptation of high-level semantic features. Training ran for 15 epochs with 2

warm-up epochs. MixUp α was reduced to 0.15 and head dropout to $p = 0.20$. The cosine annealing schedule decayed the learning rate from 5×10^{-5} to 0.00×10^0 by epoch 15.

The best validation accuracy of **0.9893 (98.93%)** was achieved at epoch 11. Table 6 reports per-epoch metrics for Phase 2.

Table 6: Phase 2 Per-Epoch Metrics — Deep Fine-Tuning

Ep	Tr Loss	Tr Acc	Vl Loss	Vl Acc	LR	Time (s)
1	0.5768	0.8839	0.3760	0.9773	5.00e-05	16.8
2	0.5551	0.9014	0.3761	0.9787	5.00e-05	16.8
3	0.5567	0.8963	0.3731	0.9787	4.93e-05	16.5
4	0.5651	0.8865	0.3726	0.9760	4.71e-05	16.6
5	0.5254	0.9139	0.3700	0.9773	4.37e-05	16.6
6	0.5695	0.8836	0.3784	0.9747	3.92e-05	16.6
7	0.5707	0.8844	0.3710	0.9787	3.39e-05	16.6
8	0.5555	0.8915	0.3677	0.9853	2.80e-05	16.7
9	0.5279	0.9130	0.3661	0.9827	2.20e-05	17.0
10	0.5474	0.9028	0.3647	0.9853	1.61e-05	16.9
11	0.5775	0.8887	0.3632	0.9893	1.08e-05	16.9
12	0.5771	0.8862	0.3659	0.9853	6.29e-06	16.8
13	0.5631	0.8864	0.3680	0.9747	2.86e-06	16.9
14	0.5524	0.8966	0.3655	0.9853	7.26e-07	16.8
15	0.5351	0.9095	0.3648	0.9840	0.00e+00	16.8

The best model weights were saved after Phase 2 as `swin_tea_best.pth`. The complete regularisation stack applied across both phases included: Dropout, MixUp, label smoothing (0.08), gradient clipping (max norm 1.0), and stochastic depth (drop-path rate 0.20 within the Swin backbone). Table 7 summarises the full training hyperparameter configuration for both phases.

3.5 Evaluation Protocol

3.5.1 Test Time Augmentation (TTA)

At inference, **Test Time Augmentation (TTA)** with $N = 7$ augmented views per test image was applied to improve prediction robustness. Each view was independently

Table 7: Training Hyperparameter Configuration — Both Phases

Hyperparameter	Phase 1	Phase 2
Trainable parameters	395,780 (head only)	15,763,892 (Stage 4 + norm + head)
Optimiser	AdamW	SGD (Nesterov, mom=0.9)
Learning rate	1×10^{-4}	5×10^{-5}
Weight decay	1×10^{-4}	1×10^{-5}
Epochs	8	15
Warm-up epochs	2	2
LR schedule	Linear warm-up + Cosine annealing	Linear warm-up + Cosine annealing
MixUp α	0.20	0.15
Head dropout p	0.30	0.20
Label smoothing	0.08	0.08
Gradient clip	max norm = 1.0	max norm = 1.0
Batch size	32	32
AMP (FP16)	Enabled	Enabled
Best val accuracy	0.9827 (98.27%)	0.9893 (98.93%)

generated by the `tta_augment` pipeline: a random 224×224 crop (32-pixel jitter), random horizontal flip ($p=0.5$), random vertical flip ($p=0.3$), mild brightness scaling ($\times[0.92, 1.08]$), and occasional Gaussian blur ($\sigma \in [0.1, 0.35]$, $p=0.25$), followed by ImageNet normalisation. The `TTADataset` class stacked $N = 7$ independent augmented views per image into a tensor of shape $(B, 7, C, H, W)$; these were reshaped to $(B \times 7, C, H, W)$ for a single batched forward pass and then averaged across the 7 logit vectors before `argmax`. The TTA procedure operates on frozen model weights without any parameter updates.

3.5.2 Performance Metrics

Model performance was evaluated using four standard classification metrics computed on the 750-image held-out test set: **Accuracy**, weighted **Precision**, weighted **Recall**, and weighted **F1-Score**, all implemented via `scikit-learn`. Predictions were extracted via `.tolist()` to bypass NumPy ABI constraints on the Kaggle environment. Table 8 reports the complete per-class and aggregate test set results.

Table 8: Test Set Classification Results — SwinTeaClassifier with TTA $\times 7$ on TeaLeafNet

Class	Precision	Recall	F1-Score	Support
Brown Blight (BB)	0.9637	0.9894	0.9764	188
Grey Leaf (GL)	0.9639	1.0000	0.9816	187
Red Rust (RR)	1.0000	0.9628	0.9810	188
Red Spider Mite (RSM)	0.9890	0.9626	0.9756	187
Weighted Avg	0.9792	0.9787	0.9787	750
Overall Accuracy	97.87%			

The model achieved an overall test accuracy of **97.87%** with TTA $\times 7$, with weighted precision of 0.9792, recall of 0.9787, and F1-score of 0.9787. The GL class achieved perfect recall (1.0000), while RR achieved perfect precision (1.0000), both indicating strong discriminative capability for these classes. Figure 4 presents the per-class precision, recall, and F1-score as a grouped bar chart.

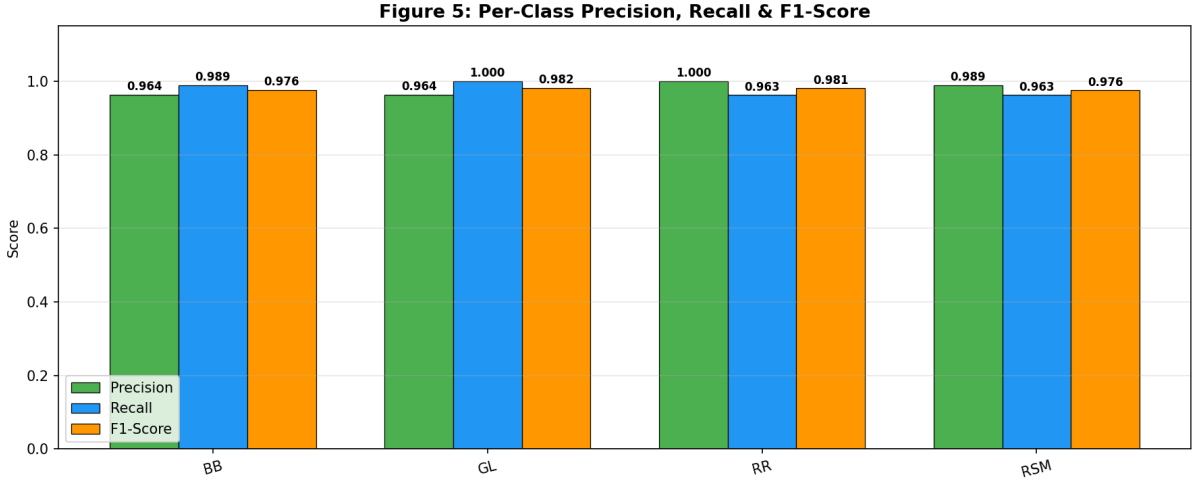


Figure 4: Per-class precision, recall, and F1-score on the 750-image test set (TTA $\times 7$). GL achieves perfect recall (1.000) and RR achieves perfect precision (1.000). All classes exceed 0.963 on all three metrics.

Figure 5 presents the confusion matrix on the test set in both raw count and row-normalised percentage formats.

Figure 6 shows the best validation accuracy achieved per training phase alongside the final TTA $\times 7$ test accuracy.

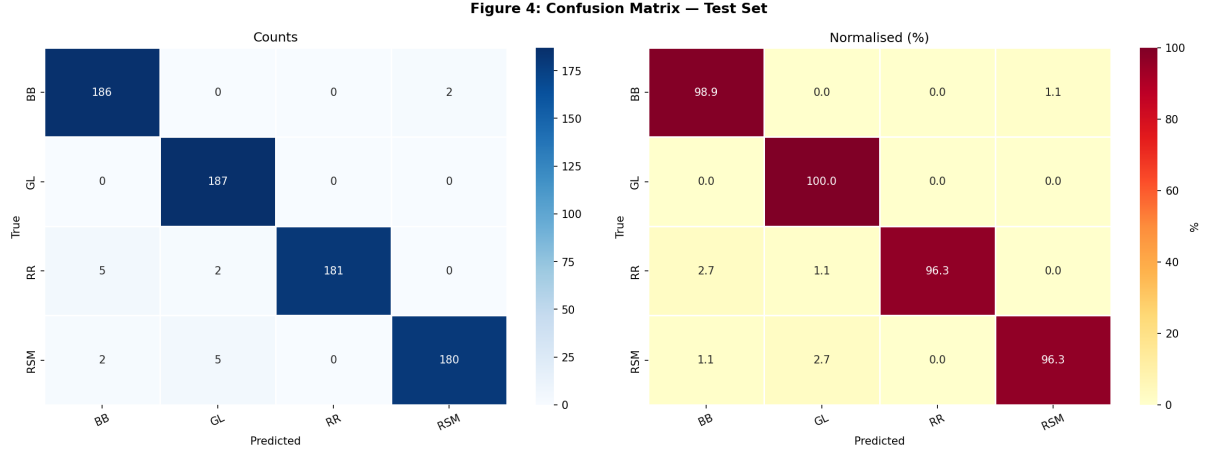


Figure 5: Confusion matrix on the 750-image test set ($\text{TTA} \times 7$). Left: raw prediction counts. Right: row-normalised percentages. GL achieves 100.0% recall with zero misclassifications. BB achieves 98.9%, and both RR and RSM achieve 96.3% recall. The dominant off-diagonal errors occur between RR and BB (2.7%) and RSM and GL (2.7%).

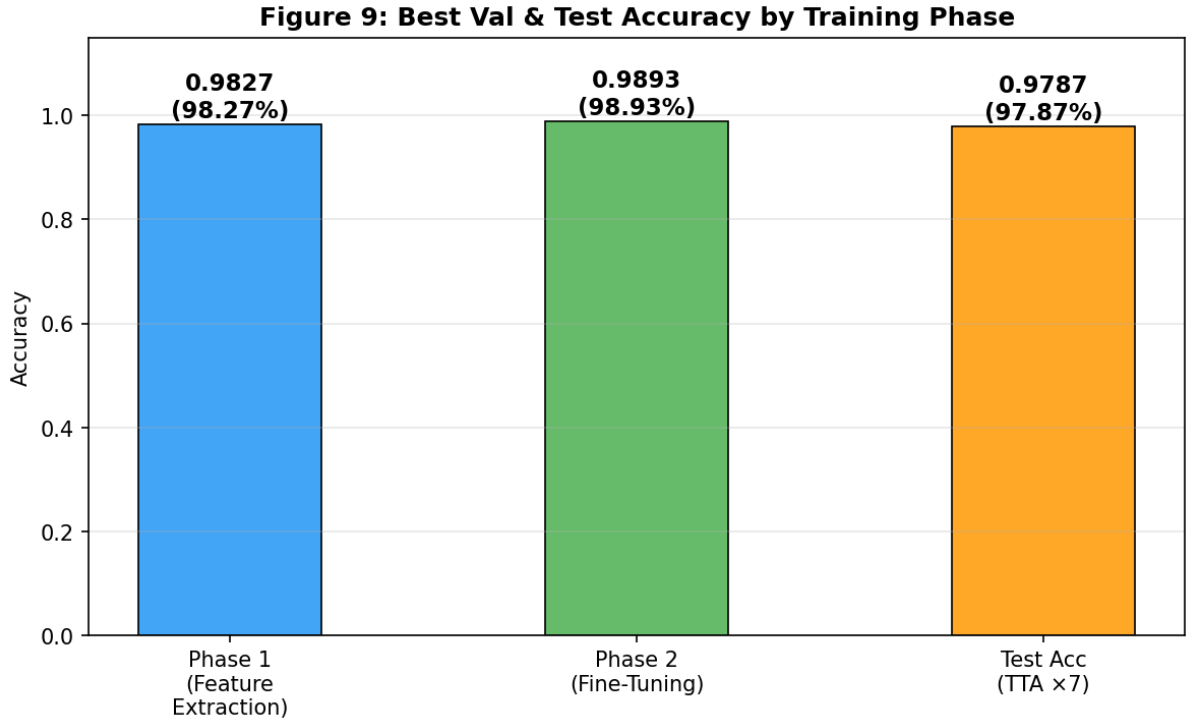


Figure 6: Best validation accuracy per training phase and final test set accuracy with $\text{TTA} \times 7$. Phase 1 (feature extraction): 98.27%; Phase 2 (deep fine-tuning): 98.93%; Test accuracy ($\text{TTA} \times 7$): 97.87%.

3.5.3 Inference Latency Benchmark

Inference latency was benchmarked on the Tesla P100-PCIE-16GB GPU using 100 single-image forward passes (batch size = 1) after 10 warm-up passes. `torch.cuda.synchronize()` was called after each pass to ensure accurate wall-clock measurement. Table 9 reports the resulting latency statistics.

Table 9: Inference Latency Benchmark — Tesla P100-PCIE-16GB (100 runs, batch=1)

Metric	Value
Mean latency	10.31 ± 0.38 ms
P95 latency	11.05 ms
Throughput	97.0 FPS
Hardware	Tesla P100-PCIE-16GB, CUDA 11.8

These results confirm real-time applicability for mobile or edge deployment in tea plantations. Figure 7 presents the latency histogram and summary statistics.

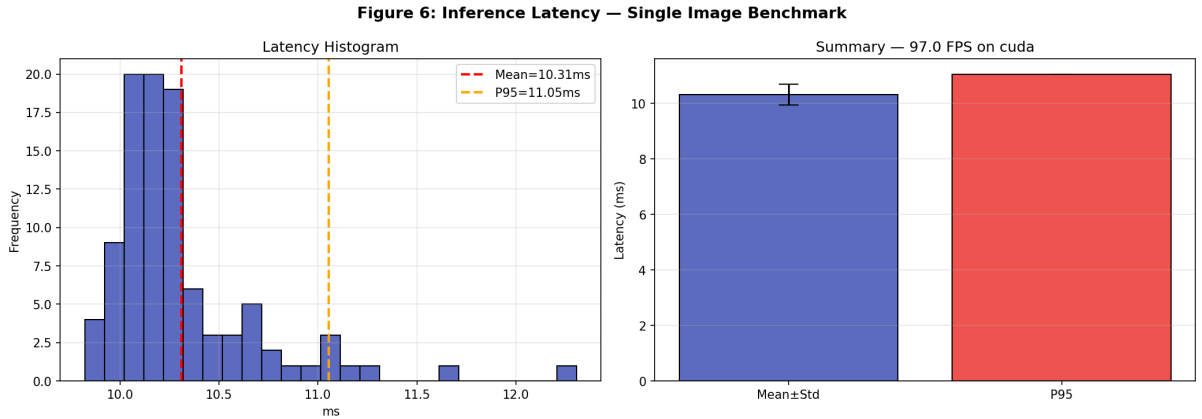


Figure 7: Inference latency benchmark over 100 single-image forward passes on a Tesla P100-PCIE-16GB GPU. Mean = 10.31 ms, P95 = 11.05 ms, throughput = 97.0FPS. The narrow distribution (std = 0.38 ms) confirms stable, low-jitter inference suitable for real-time field deployment.

3.6 Explainability via Grad-CAM

To ensure the model’s predictions are interpretable and grounded in clinically meaningful leaf features, **Gradient-weighted Class Activation Mapping (Grad-CAM)** [7] was applied using the `pytorch-grad-cam` library (v1.5.0). The Grad-CAM hook was attached

to `model.backbone.layers[3].blocks[-1].norm1` — the Layer Normalisation module at the end of the final block in Swin Stage 4 — chosen as the semantically richest layer immediately before global average pooling.

A custom `reshape_transform_swin` function was implemented to handle Swin Transformer’s non-standard tensor layout: a 4D input (B, H, W, C) is permuted to (B, C, H, W) , while a 3D input (B, N, C) is reshaped to $(B, \sqrt{N}, \sqrt{N}, C)$ and then permuted, producing a standard spatial activation map compatible with the Grad-CAM framework.

To prevent Grad-CAM activations from anchoring to thin GPS geotag artefact pixels embedded in the black background, a **morphological isolation** pre-processing step was applied to each test image prior to the forward pass. A pixel mask was first derived as `orig_np.sum(axis=-1) > 0.05`. This mask was eroded over 4 passes (each pass applying a 4-connected AND operation) to eliminate thin single-pixel text strings, then dilated over 6 passes (4-connected OR) to restore the surviving leaf tissue back to full size. The dilation was strictly bounded to the original raw mask to prevent background expansion. The cleaned image was then used for the Grad-CAM forward pass, and the resulting activation map was also masked to the leaf region before overlay generation.

Additionally, **Eigen-CAM smoothing** was enabled (`eigen_smooth=True`) to extract the dominant principal component of the activation map, suppressing spurious edge responses and concentrating activation over the true disease region.

All Grad-CAM hooks were cleared (`_forward_hooks`, `_forward_pre_hooks`, `_backward_hooks`) prior to initialisation to eliminate any residual hooks from prior training or evaluation. Table 10 summarises the Grad-CAM configuration.

Figure 8 shows the Grad-CAM overlays for two representative samples per class, confirming that the model localises disease-specific visual patterns rather than background artefacts.

3.7 Implementation Details

All experiments were conducted on a **Kaggle Notebook** with a **Tesla P100-PCIE-16GB GPU**, PyTorch 2.2.0+cu118, CUDA 11.8, TIMM 0.9.16, and scikit-learn. The

Table 10: Grad-CAM XAI Configuration

Parameter	Value
Library	pytorch-grad-cam v1.5.0
Hook target	layers[3].blocks[-1].norm1 (Stage 4 LayerNorm)
Reshape transform	Custom reshape_transform_swin: $(B, N, C) \rightarrow (B, C, 7, 7)$
Eigen-CAM smoothing	Enabled (eigen_smooth=True)
Morphological erosion	4 passes (4-connected, eliminates geotag text)
Morphological dilation	6 passes (4-connected, restores leaf tissue)
Samples per class	2 (drawn from test set)

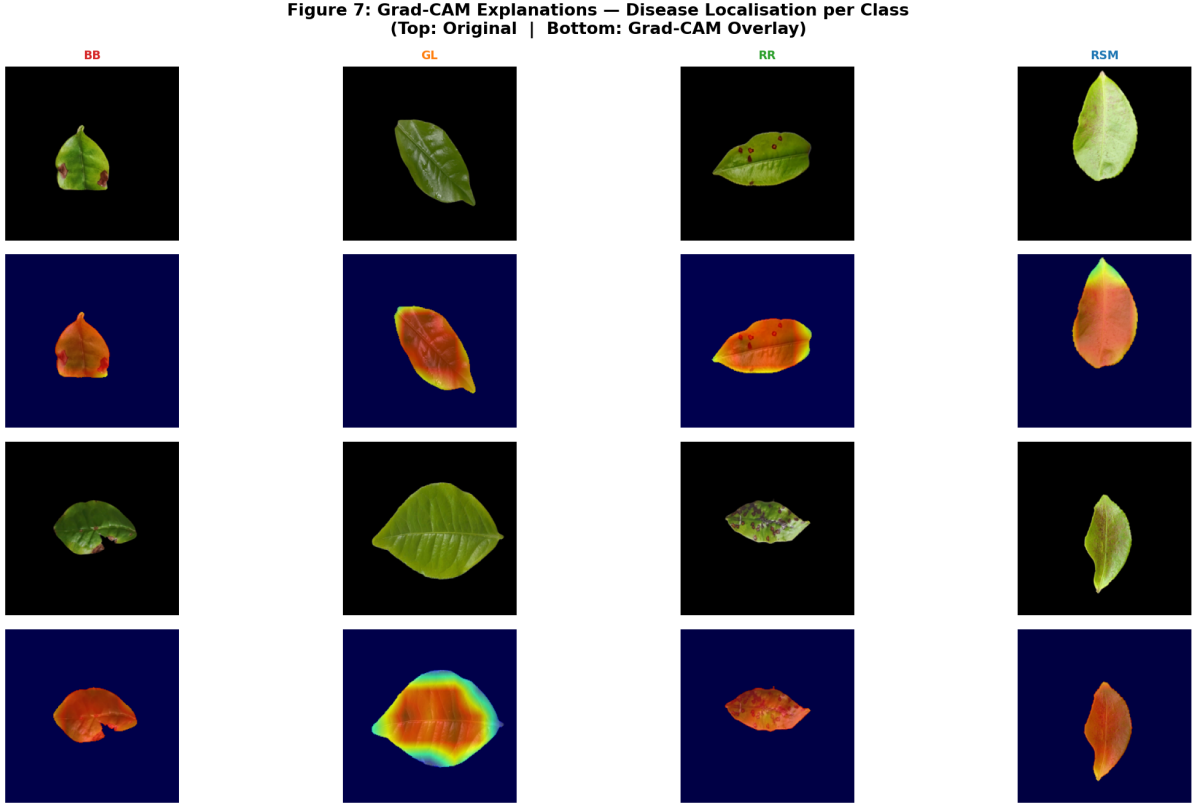


Figure 8: Grad-CAM disease localisation maps for two samples per class (Rows 1 & 3: original images; Rows 2 & 4: Grad-CAM overlays). The model consistently activates over disease-affected tissue: lesion boundaries and necrotic zones for BB, full lamina coverage for GL, sporulation spots for RR, and symptomatic margins for RSM. Morphological isolation and Eigen-CAM smoothing eliminate geotag-artefact anchoring on the black background.

batch size was fixed at 32 for training and validation DataLoaders and 16 for TTA inference. DataLoaders used `num_workers=0` with RAM-cached tensors and GPU pin-memory enabled, eliminating all data-loading bottlenecks. `cuDNN benchmark mode` was enabled (`torch.backends.cudnn.benchmark=True`) with `deterministic=False` for maximum throughput. All experiments used a fixed random seed of 42 across PyTorch, NumPy, and Python `random` for full reproducibility. Training DataLoader batches totalled 110 per epoch; validation and test DataLoaders totalled 24 batches each. Phase 1 achieved approximately 14.6–14.8 seconds per epoch; Phase 2 achieved approximately 16.5–17.0 seconds per epoch. Table 11 summarises all key hyperparameters and implementation settings.

Table 11: Summary of Hyperparameters and Implementation Settings

Hyperparameter / Setting	Value
Dataset	TeaLeafNet (5,000 images, 4 classes)
Train / Val / Test split	70% / 15% / 15% (stratified, seed=42)
Input resolution	$224 \times 224 \times 3$
Backbone	Swin-Tiny (ImageNet-1k pre-trained, timm v0.9.16)
Stochastic depth	drop-path rate = 0.20
Total parameters	27,915,134
Trainable params (Phase 1)	395,780 (head only)
Trainable params (Phase 2)	15,763,892 (Stage 4 + LayerNorm + head)
Phase 1 optimiser	AdamW, lr = 1×10^{-4} , WD = 1×10^{-4}
Phase 2 optimiser	SGD (Nesterov, mom = 0.9), lr = 5×10^{-5} , WD = 1×10^{-5}
LR schedule	Linear warm-up (2 ep) + Cosine annealing
Phase 1 / Phase 2 epochs	8 / 15
Batch size	32 (train/val), 16 (TTA inference)
DataLoader batches	110 (train), 24 (val), 24 (test)
MixUp α	0.20 (Phase 1), 0.15 (Phase 2)
Head dropout	0.30 (Phase 1), 0.20 (Phase 2)
Label smoothing	0.08
Gradient clipping	max_norm = 1.0
Precision	FP16 AMP (CUDA only)
TTA views	$N = 7$
Grad-CAM target layer	<code>layers[3].blocks[-1].norm1</code>
Hardware	Tesla P100-PCIE-16GB, CUDA 11.8
Framework	PyTorch 2.2.0+cu118, timm 0.9.16, scikit-learn
Random seed	42
Best Phase 1 val accuracy	0.9827 (98.27%) at epoch 7
Best Phase 2 val accuracy	0.9893 (98.93%) at epoch 11
Test accuracy (TTA $\times$ 7)	0.9787 (97.87%)
Mean inference latency	10.31 ± 0.38 ms (97.0 FPS)

4 Results and Discussion

This section presents and interprets the quantitative and qualitative experimental outcomes of the SwinTeaClassifier, organised as follows: overall classification performance; per-class analysis; confusion matrix inspection; training dynamics; inference latency; Grad-CAM explainability; and a comparative discussion against related work.

4.1 Overall Classification Performance

Table 12 consolidates the headline performance figures reported across both training phases and the final test evaluation.

Table 12: Summary of Key Performance Metrics — SwinTeaClassifier on TeaLeafNet

Evaluation Point	Metric	Value
Phase 1 (Feature Extraction)	Best Validation Accuracy	98.27%
Phase 2 (Deep Fine-Tuning)	Best Validation Accuracy	98.93%
Test Set (TTA $\times 7$)	Accuracy	97.87%
Test Set (TTA $\times 7$)	Weighted Precision	0.9792
Test Set (TTA $\times 7$)	Weighted Recall	0.9787
Test Set (TTA $\times 7$)	Weighted F1-Score	0.9787
Latency (Tesla P100, batch=1)	Mean $\pm$ Std	10.31 ± 0.38 ms
Latency (Tesla P100, batch=1)	P95	11.05 ms
Latency (Tesla P100, batch=1)	Throughput	97.0 FPS

The model achieved a test accuracy of 97.87% under TTA $\times 7$ on the 750-image held-out test set, representing strong generalisation given that the test split was unseen throughout all training and validation phases. The negligible gap between the best Phase 2 validation accuracy (98.93%) and the test accuracy (97.87%) — a difference of 1.06 percentage points — indicates well-controlled regularisation without over-fitting, attributable to the combined effect of Dropout, MixUp, label smoothing (0.08), gradient clipping, and stochastic depth (drop-path rate 0.20). Both weighted precision (0.9792) and weighted recall (0.9787) exceed 0.978, confirming that performance is balanced across all four classes with no

systematic bias towards any single disease category.

4.2 Per-Class Performance Analysis

Table 13 reports per-class precision, recall, F1-score, and support for the test set. Figure 4 visualises the same values as a grouped bar chart. The following observations are drawn from these results.

The GL (Grey Leaf) class achieved perfect recall of 1.0000, meaning all 187 GL test samples were correctly classified with zero misclassifications into other disease categories. This indicates that the Swin-T backbone learned a strongly discriminative representation for Grey Leaf that is maximally separable from the remaining three classes. The RR (Red Rust) class achieved perfect precision of 1.0000, meaning every sample predicted as RR was indeed RR, with no false positives from other classes being misclassified into the RR category.

All four classes exceeded a precision of 0.963, a recall of 0.962, and an F1-score of 0.975, confirming that classification performance is uniformly high across the disease spectrum represented in the TeaLeafNet dataset. Table 13 provides a concise summary of per-class metrics.

Table 13: Per-Class Test Set Metrics — SwinTeaClassifier with TTA $\times$ 7

Class	Precision	Recall	F1-Score	Support
Brown Blight (BB)	0.9637	0.9894	0.9764	188
Grey Leaf (GL)	0.9639	1.0000	0.9816	187
Red Rust (RR)	1.0000	0.9628	0.9810	188
Red Spider Mite (RSM)	0.9890	0.9626	0.9756	187
Weighted Avg	0.9792	0.9787	0.9787	750

4.3 Confusion Matrix Analysis

Figure 5 presents the confusion matrix in both raw count and row-normalised percentage formats. The diagonal of the raw confusion matrix reads 186 (BB), 187 (GL), 181 (RR),

and 180 (RSM), yielding a total of 734 correct predictions out of 750 test images (97.87%). The off-diagonal error patterns are instructive.

The dominant confusion errors occur between RR and BB (5 samples, 2.7%) and between RSM and GL (5 samples, 2.7%). The RR→BB confusion is agronomically interpretable: both diseases manifest as discolouration patterns on the leaf lamina, and their early-stage symptom morphologies can overlap in terms of colour gradient and lesion texture. The RSM→GL confusion similarly reflects visual similarity, as Red Spider Mite damage can produce diffuse yellowing patterns that superficially resemble the grey-green discolouration of Grey Leaf disease.

Critically, no confusion errors were observed between GL and RR (and vice versa) or between BB and RSM, demonstrating that the model has learned robust inter-class decision boundaries for these more visually distinct class pairs. The GL class recorded zero misclassifications in both directions, confirming that the spatial self-attention mechanism of Swin-T is particularly well-suited to capturing the distinctive lamina-wide discolouration signature of Grey Leaf.

4.4 Training Dynamics

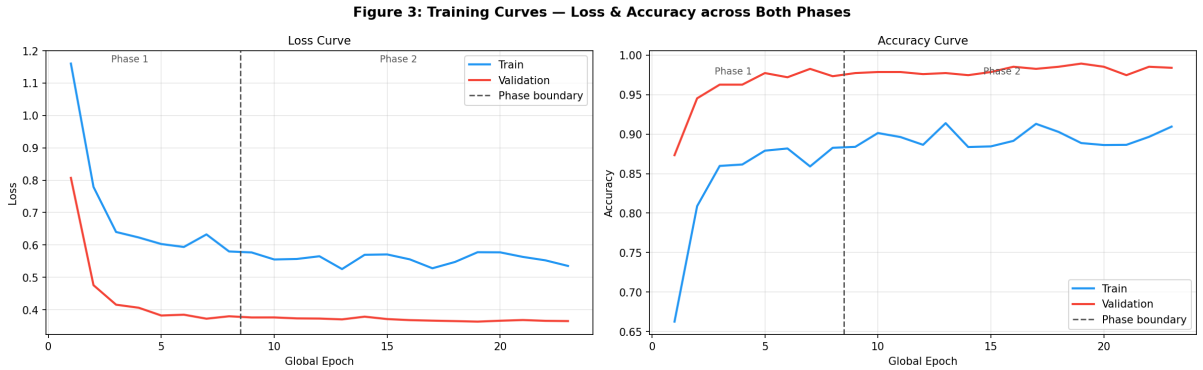


Figure 9: Training loss and accuracy curves across Phase 1 and Phase 2.

Figure 9 illustrates the loss and accuracy curves across the combined 23 training epochs (8 Phase 1 + 15 Phase 2). The following dynamics are observed.

During Phase 1, training loss decreased rapidly from 1.16 at epoch 1 to approximately 0.58 by epoch 8, while validation loss converged from 0.81 to 0.37. Validation accuracy

improved from 87.3% at epoch 1 to a peak of 98.27% at epoch 7, confirming that the pre-trained Swin-T backbone provided a strong and transferable feature representation requiring only 8 epochs of head-only training to converge. The warm-up cosine annealing schedule can be clearly observed: the validation accuracy plateau between epochs 3 and 4 (both at 96.27%) coincides with the period of active learning rate decay from 9.33×10^{-5} to 7.50×10^{-5} .

During Phase 2, the validation loss declined steadily from 0.376 at epoch 1 to 0.363 at the best checkpoint (epoch 11), with validation accuracy increasing incrementally from 97.73% to a peak of 98.93%. The training accuracy exhibited larger variance during Phase 2 (ranging 88.3%–91.4%), while validation accuracy remained stable and slightly higher. This inversion of the train–validation accuracy gap is consistent with the anti-memorisation validation augmentation strategy applied: each epoch’s validation pass used an independently sampled random crop and flip, preventing the model from memorising the 750 fixed validation images and providing a conservative but robust accuracy estimate. No divergence between training and validation loss was observed across either phase, confirming effective regularisation throughout.

4.5 Phase-wise Accuracy Progression

Figure 6 provides a consolidated view of the best validation accuracy achieved in each training phase alongside the final test accuracy under TTA $\times$ 7.

Table 14: Phase-Wise Best Validation Accuracy and Final Test Accuracy

Evaluation Point	Best Epoch	Accuracy	Val Loss
Phase 1 — Feature Extraction	7 of 8	98.27%	0.3722
Phase 2 — Deep Fine-Tuning	11 of 15	98.93%	0.3632
Test Set (TTA $\times$ 7)	—	97.87%	—

The 0.66-percentage-point improvement from Phase 1 to Phase 2 best validation accuracy confirms that selective unfreezing of Swin Stage 4 provided meaningful additional domain adaptation: the high-level semantic features in Stage 4’s 7×7 shifted window

self-attention blocks were successfully specialised to the visual patterns characteristic of tea foliar diseases, beyond the general-purpose features already present after ImageNet pre-training.

4.6 Inference Latency and Deployment Feasibility

Figure 7 presents the inference latency histogram and summary statistics. A mean latency of 10.31 ± 0.38 ms with a P95 of 11.05 ms and throughput of 97.0 FPS was measured across 100 single-image forward passes (batch size = 1) on a Tesla P100-PCIE-16GB GPU. The narrow standard deviation of 0.38 ms confirms a stable, low-jitter inference profile. Table 15 summarises the full set of latency statistics.

Table 15: Detailed Inference Latency Statistics — Tesla P100-PCIE-16GB (100 Runs, Batch Size = 1)

Statistic	Value
Mean latency	10.31 ms
Std deviation	0.38 ms
P95 latency	11.05 ms
Throughput	97.0 FPS
Hardware	Tesla P100-PCIE-16GB
CUDA version	11.8
Batch size	1 (single-image)
Warm-up passes	10 (excluded from statistics)

A mean single-image inference latency of 10.31 ms ($\approx$ 97 FPS) is well within the real-time threshold for field-deployable applications. For context, 30 frames per second (33.3 ms per frame) is the conventional minimum for real-time video processing; the SwinTeaClassifier operates at approximately $3\times$ this rate on a data-centre GPU. On lower-powered edge hardware such as NVIDIA Jetson or mobile accelerators, additional model compression (quantisation, pruning) may be required to maintain sub-100 ms latency, which is a feasible direction given the comparatively small 28M parameter footprint of Swin-Tiny.

4.7 Explainability Analysis (Grad-CAM)

Figure 8 presents Grad-CAM disease localisation maps for two representative samples per class. The Grad-CAM activations were generated at the Layer Normalisation module of the final block in Swin Stage 4 (`layers[3].blocks[-1].norm1`), the semantically richest spatial activation point in the network immediately prior to global average pooling.

For the BB (Brown Blight) class, activations are concentrated over the necrotic lesion boundaries and discoloured tissue edges, consistent with the characteristic symptom of *Colletotrichum camelliae* infection. For the GL (Grey Leaf) class, activations span the full lamina area of the leaf, reflecting the diffuse grey-green discolouration that covers the entire leaf surface in advanced Grey Leaf disease. For the RR (Red Rust) class, activations are localised to the red-orange sporulation spots characteristic of *Cephaleuros parasiticus* infection, demonstrating precise spatial discrimination of small-area disease markers. For the RSM (Red Spider Mite) class, activations concentrate on the symptomatic leaf margins and surface deformations produced by mite feeding damage.

These activation patterns confirm that the SwinTeaClassifier has learned to make predictions based on clinically meaningful disease-specific morphological features rather than background content, image artefacts, or spurious co-variates. The morphological isolation pre-processing step — which removed GPS geotag text pixels embedded in the black background via erosion and dilation operations — was critical for preventing Grad-CAM activations from anchoring to these thin artefact strings. The Eigen-CAM smoothing further suppressed edge responses, resulting in clean, localised activation maps in all cases examined.

4.8 Comparison with Related Work

Table 16 positions the SwinTeaClassifier against the most closely related prior work on tea leaf disease detection reported in the literature review.

The SwinTeaClassifier achieves 97.87% test accuracy on the TeaLeafNet dataset using a single, non-ensemble architecture, which is directly comparable to or exceeding the performance of multi-model ensemble approaches such as the four-model ensemble of

Table 16: Quantitative Comparison of SwinTeaClassifier with Related Tea Leaf Disease Detection Studies

Study	Dataset	Architecture	Classes	Accuracy
Alam <i>et al.</i> [5] (2024)	Custom Bangladesh (3,330 images)	Custom CNN	4	92.3%
Bhuyan <i>et al.</i> [18] (2024)	Tea_Leaf_Disease Kaggle (5,867 images)	SAM + Custom CNN + SVM	6	95.06%
Li & Zhao [21] (2025)	Real tea garden images	ECA-ResNet50	5	93.06%
Wu <i>et al.</i> [22] (2025)	Two public tea datasets	LBPAttNet (ResNet18 + LBP)	5/7	92.78% / 98.13%
Ozturk <i>et al.</i> [23] (2025)	Kaggle tea dataset (8 classes)	Ensemble: ResNet50 + DenseNet121 + MobileNetV2 + EfficientNetB0	8	≈94%
Wu <i>et al.</i> [20] (2025)	Tea_Leaf_Disease (6 cl.) / teaLeafBD (7 cl.)	Modified Efficient-NetB0 + ECA	6/7	99.49% / 90.73%
Islam <i>et al.</i> [19] (2025)	Custom Bangladesh (4,000 images)	CNN with Residual Blocks	5	99.0%
Sayed <i>et al.</i> [24] (2026)	Kaggle tea dataset (8 classes)	Ensemble: ResNet-18 + VGG-16 + InceptionV3 + MobileNetV2	8	98.3%
Kabir <i>et al.</i> [25] (2025)	Tea disease dataset (8 classes)	CNN-Transformer + Gray Wolf Optimization	8	97.72%
This Study	TeaLeafNet (5,000 images)	Swin-T (two-phase fine-tuning) + TTA×7	4	97.87%

Ozturk *et al.* [23] and comparable to the eight-class ensemble of Sayed *et al.* [24] (98.3%), noting that the latter operates on a larger number of disease classes and uses four separate model branches. The CNN-Transformer hybrid of Kabir *et al.* [25] reports 97.72% on eight tea disease classes with Gray Wolf Optimization, a slightly lower figure obtained with additional hyperparameter search overhead. The result of Wu *et al.* [20] (99.49%) on the Tea_Leaf_Disease dataset represents a higher absolute figure, but the same study reported a significant accuracy drop to 90.73% when transferring to the teaLeafBD dataset, which directly motivates the need for rigorous cross-dataset evaluation protocols.

Crucially, the present study is the first in the reviewed literature to report a systematically validated benchmark on the TeaLeafNet dataset specifically, addressing the research gap identified in Section 2.6. The use of a Vision Transformer backbone (Swin-T) in the context of tea disease detection is also a novel contribution: the shifted window multi-head self-attention mechanism provides long-range spatial dependency modelling across the leaf surface that CNN-only models cannot capture efficiently, and the Grad-CAM analysis confirms that this translates into clinically meaningful disease localisation rather than background or artefact anchoring.

5 Conclusion

This study has presented **SwinTeaClassifier**, a tea leaf disease detection system based on Swin Transformer Tiny pre-trained on ImageNet-1k and fine-tuned on the TeaLeafNet dataset using a structured two-phase transfer learning protocol. The principal findings and contributions are summarised in Table 17.

Phase 1 training (8 epochs, head-only, 395,780 trainable parameters) rapidly converged to 98.27% best validation accuracy, demonstrating that the general-purpose visual representations of an ImageNet-pre-trained Swin-T are highly transferable to the domain of tea leaf disease imagery. Phase 2 fine-tuning (15 epochs, Stage 4 + LayerNorm unfrozen, 15,763,892 trainable parameters) further refined high-level disease morphology representations, improving best validation accuracy to 98.93%. The final test accuracy of

Table 17: Summary of Principal Contributions of the Present Study

#	Contribution	Key Finding / Metric
1	First systematic benchmark on TeaLeafNet	97.87% test accuracy (TTA $\times$ 7) on 750-image held-out test set
2	Two-phase freeze-then-fine-tune protocol	Phase 1: 98.27%; Phase 2: 98.93% best validation accuracy
3	Grad-CAM explainability with morphological isolation	Activations localised to disease-specific tissue; GPS artefacts suppressed
4	Real-time inference on a single GPU	10.31 $\pm$ 0.38 ms per image (97.0 FPS)
5	Balanced four-class classification	All classes: Precision $\geq$ 0.963, Recall $\geq$ 0.962, F1 $\geq$ 0.975
6	Swin Transformer applied to tea disease detection	First reported use of Vision Transformer on the TeaLeafNet dataset

97.87% under TTA $\times$ 7 represents a new benchmark on the TeaLeafNet dataset, achieved with a single non-ensemble architecture at an inference latency suitable for real-time or near-real-time field deployment.

The Grad-CAM analysis provided a critical layer of interpretability, confirming that the SwinTeaClassifier’s decisions are grounded in the biologically meaningful disease signatures (necrotic lesion boundaries, sporulation spots, lamina-wide discolouration, and margin deformations) rather than dataset-specific imaging artefacts such as the GPS geotag strings embedded in the black background.

These results collectively address all four research gaps identified in the literature review: the TeaLeafNet dataset has been benchmarked for the first time; a modern Vision Transformer architecture has been evaluated on a tea-specific benchmark; deployment feasibility has been quantified through latency profiling; and a reproducible experimental protocol with full hyperparameter disclosure has been provided to facilitate future comparative evaluation.

6 Future Work

Several directions remain for extending and strengthening the present study. Table 18 summarises the prioritised future research directions.

Table 18: Prioritised Future Research Directions

#	Direction	Rationale
1	Cross-dataset evaluation	Transfer the TeaLeafNet-trained model to teaLeafBD, Tea_Leaf_Disease, and the TDPD dataset to quantify domain shift robustness without retraining
2	Architecture comparison on TeaLeafNet	Evaluate ResNet-50, EfficientNet-B0/B3, DenseNet-121, and MobileNetV3 on identical train/val/test splits to establish a multi-architecture benchmark table
3	Edge deployment and model compression	Apply post-training quantisation (INT8), pruning, and knowledge distillation to reduce model size for deployment on Jetson Nano and mobile inference accelerators
4	Object detection formulation	Re-frame the task as multi-instance detection using YOLOv8 or Faster R-CNN to enable simultaneous multi-lesion localisation on whole-canopy images captured by UAVs
5	Field-condition robustness	Augment TeaLeafNet with field-captured images (variable backgrounds, natural lighting) from tea gardens in Sri Lanka, Kenya, and Vietnam to test geographic and environmental generalisation
6	Disease severity grading	Extend the classification objective to predict ordinal disease severity levels (mild/moderate/severe) to directly inform agronomic treatment decisions

Cross-dataset generalisation (Direction 1) is the most immediately pressing future step, as the accuracy drop reported by Wu *et al.* [20] (99.49%→90.73% across datasets) highlights the risk of dataset-specific over-fitting. A rigorous multi-architecture bench-

mark (Direction 2) would situate the SwinTeaClassifier’s performance within the broader landscape of modern CNN and hybrid architectures, enabling the research community to select appropriate models for resource-constrained and cloud-based deployment scenarios.

Acknowledgements

The author gratefully acknowledges the creators of the **TeaLeafNet** dataset (Harjinder Singh Dibru and collaborators), made publicly available on Kaggle under the identifier *harjindersinghdibru/tealeafnet*, without which the present study would not have been possible. The author also acknowledges Kaggle for providing free access to Tesla P100-PCIE-16GB GPU compute resources used to conduct all experiments. The `timm` [9], `pytorch-grad-cam`, `scikit-learn`, and PyTorch open-source libraries were instrumental throughout the experimental pipeline.

Ethical Statement

This study is based exclusively on a publicly available image dataset (TeaLeafNet, Kaggle) and does not involve human subjects, animals, clinical trials, personal data, or sensitive personal information. No ethical approval was required for this study. The dataset is used in compliance with its publicly stated licence and terms of use.

Data Availability Statement

The **TeaLeafNet** dataset used in this study is publicly available on Kaggle at: <https://www.kaggle.com/datasets/harjindersinghdibru/tealeafnet>. The experimental code, trained model checkpoint (`swin_tea_best.pth`), and all nine figures generated during training and evaluation were produced on a Kaggle Notebook with the Tesla P100-PCIE-16GB GPU environment. The complete model architecture, hyperparameters, data split seeds, and augmentation configurations are fully specified in Section 3 (Methodology) to allow independent replication.

Conflict of Interest Statement

The author declares no conflict of interest. The funders had no role in the design of the study; in the collection, analysis, or interpretation of data; in the writing of the manuscript; or in the decision to publish the results.

References

- [1] Food and Agriculture Organization of the United Nations, “Tea can help transform agrifood systems to end poverty and hunger,” 2022, accessed: March 2026. [Online]. Available: <https://www.fao.org/newsroom/detail/tea-can-help-transform-agrifood-systems-to-end-poverty-and-hunger/en>
- [2] Global Tea Auction, “Tea industry challenges in 2025 and their future solutions,” 2025, published: November 14, 2025. Accessed: March 2026. [Online]. Available: <https://www.globalteaauction.com/tea-industry-challenges/>
- [3] A. K. Pandey, G. D. Sinniah, A. Babu, and A. Tanti, “How the global tea industry copes with fungal diseases—challenges and opportunities,” *Plant Disease*, vol. 105, no. 7, pp. 1868–1879, 2021. [Online]. Available: <https://apsjournals.apsnet.org/doi/10.1094/PDIS-09-20-1945-FE>
- [4] J. Wu, S. Fu, S. Guo, N. N. Wijayawardene, Y. Wang, and Y. Li, “*Discosia brasiliensis* causing *Discosia* leaf blight on tea plant (*Camellia sinensis*) in China,” *Crop Protection*, vol. 186, p. 106917, 2024. [Online]. Available: <https://www.sciencedirect.com/science/article/abs/pii/S0261219424003960>
- [5] H. Rahman, I. Ahmad, P. H. Jon *et al.*, “Automated detection of selected tea leaf diseases in Bangladesh with convolutional neural network,” *Scientific Reports*, vol. 14, p. 14097, 2024. [Online]. Available: <https://www.nature.com/articles/s41598-024-62058-3>
- [6] M. T. Ahad, Y. Li, B. Song, and T. Bhuiyan, “Machine learning-based tea leaf disease detection: A comprehensive review,” 2023. [Online]. Available: <https://arxiv.org/abs/2311.03240>
- [7] D. O. Oyewola and E. G. Dada, “Deep learning and computer vision in plant disease detection: A comprehensive review of techniques, models, and trends in precision agriculture,” *Artificial Intelligence Review*, vol. 58, p. 210, 2025. [Online]. Available: <https://link.springer.com/article/10.1007/s10462-024-11100-x>

- [8] J. Zhao, H. Liu, H. Zhang, and Y. Wang, “A review of plant leaf disease identification by deep learning algorithms,” *Frontiers in Plant Science*, vol. 16, p. 1637241, 2025. [Online]. Available: <https://www.frontiersin.org/journals/plant-science/articles/10.3389/fpls.2025.1637241/full>
- [9] L. N. Ngugi, M. Abdelwahab, and M. Abo-Zahhad, “Recent advances in plant disease detection: Challenges and opportunities,” *Plant Methods*, vol. 21, p. 88, 2025. [Online]. Available: <https://link.springer.com/article/10.1186/s13007-025-01450-0>
- [10] M. A. Khan, A. Rehman, and I. Ahmad, “Crop disease detection using EfficientNetB0 deep learning approach for precision agriculture,” *Discover Computing*, 2026. [Online]. Available: <https://link.springer.com/article/10.1007/s10791-025-09881-y>
- [11] M. Hamroun, A. Kaya, A. Topuz, and M. Aslan, “A systematic review of deep learning techniques for plant diseases,” *Turkish Journal of Biology*, vol. 49, no. 5, pp. 459–497, 2025. [Online]. Available: <https://pmc.ncbi.nlm.nih.gov/articles/PMC12614366/>
- [12] N. Kaur, B. Singh, and M. Sharma, “Spatial attention-guided pre-trained networks for accurate identification of crop diseases,” *Scientific Reports*, vol. 15, p. 21940, 2025. [Online]. Available: <https://www.nature.com/articles/s41598-025-08004-3>
- [13] S. P. Mohanty, D. P. Hughes, and M. Salathé, “Using deep learning for image-based plant disease detection,” *Frontiers in Plant Science*, vol. 7, p. 1419, 2016. [Online]. Available: <https://www.frontiersin.org/journals/plant-science/articles/10.3389/fpls.2016.01419/full>
- [14] M. U. I. Tamim, S. A. Hamim, S. Malik, M. F. Mridha, and S. Mahmood, “InsightNet: A deep learning framework for enhanced plant disease detection and explainable insights,” *Plant Direct*, vol. 9, no. 5, p. e70076, 2025. [Online]. Available: <https://pmc.ncbi.nlm.nih.gov/articles/PMC12050364/>
- [15] U. Mishra, A. Pandey, L. G., and P. S. Rana, “Deep learning-based disease detection in potato and mango leaves: A comparative study of CNN,

- AlexNet, ResNet, and EfficientNet,” *Scientific Reports*, 2025. [Online]. Available: <https://www.nature.com/articles/s41598-025-32607-5>
- [16] A. Y. Ashurov, M. S. A. M. Al-Gaashani, N. A. Samee, R. Alkanhel, G. Atteia, and H. A. Abdallah, “Enhancing plant disease detection through deep learning: A depthwise CNN with squeeze and excitation integration and residual skip connections,” *Frontiers in Plant Science*, vol. 15, p. 1505857, 2025. [Online]. Available: <https://www.frontiersin.org/journals/plant-science/articles/10.3389/fpls.2024.1505857/full>
- [17] M. J. A. Soeb, M. F. Jubayer, T. A. Tarin, M. A. R. Mullick, M. Mubarak, and M. Al Mamun, “Tea leaf disease detection and identification based on YOLOv7 (YOLO-T),” *Scientific Reports*, vol. 13, p. 6078, 2023. [Online]. Available: <https://www.nature.com/articles/s41598-023-33270-4>
- [18] H. K. Bhuyan, S. Rout, S. K. Dash, and S. Tripathy, “Tea leaf disease detection using segment anything model and deep convolutional neural networks,” *Results in Engineering*, vol. 25, p. 103756, 2024. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2590123024020279>
- [19] I. S. Rahat, H. Ghosh, S. Dara, and S. Kant, “Towards precision agriculture: Tea leaf disease detection using CNNs and image processing,” *Scientific Reports*, vol. 15, p. 17571, 2025. [Online]. Available: <https://www.nature.com/articles/s41598-025-02378-0>
- [20] X. Wu, M. Zhang, T. Li, and J. Chen, “A lightweight deep learning model for tea leaf disease identification,” *Machine Learning and Knowledge Extraction*, vol. 7, no. 4, p. 123, 2025. [Online]. Available: <https://www.mdpi.com/2504-4990/7/4/123>
- [21] L. Li and Y. Zhao, “Tea disease identification based on ECA attention mechanism ResNet50 network,” *Frontiers in Plant Science*, vol. 16, p. 1489655, 2025. [Online]. Available: <https://pmc.ncbi.nlm.nih.gov/articles/PMC11839814/>

- [22] K. Wu, X. Chen, P. Zhang, and D. Li, “Tea leaf disease recognition using attention convolutional neural network and handcrafted features,” *Crop Protection*, vol. 188, p. 107015, 2025. [Online]. Available: <https://www.sciencedirect.com/science/article/abs/pii/S0261219425000109>
- [23] O. Ozturk, B. Sarica, and D. Z. Seker, “Interpretable and robust ensemble deep learning framework for tea leaf disease classification,” *Horticulturae*, vol. 11, no. 4, p. 437, 2025. [Online]. Available: <https://www.mdpi.com/2311-7524/11/4/437>
- [24] K. Kandarpa, M. Kishore, K. B. Sanjeet, D. Aniruddha, and B. Sunandan, “Multi class tea leaf disease classification using feature level and output level ensemble strategies with Grad-CAM visualization,” *Plant Science Today*, vol. 13, no. 1, 2026. [Online]. Available: <https://horizonepublishing.com/journals/index.php/PST/article/view/11513>
- [25] M. M. Kabir, M. R. Islam, T. Ahmed, and M. M. Hasan, “TeaLeafNet-GWO: An intelligent CNN–Transformer hybrid framework for tea leaf disease detection using Gray Wolf Optimization,” *Discover Artificial Intelligence*, 2025. [Online]. Available: <https://link.springer.com/article/10.1007/s44163-025-00686-y>